COURSE NAME

OBJECT ORIENTED ANALYSIS AND DESIGN

CSC 2210

(UNDERGRADUATE)

UNIFIED MODELING LANGUAGE

CHAPTER 3

CLASS DIAGRAM

MOHAIMEN-BIN-NOOR
ASSISTANT PROFESSOR, CS
mohaimen.niloy@aiub.edu

CRC – CLASS RESPONSIBILITY COLLABORATORS

- CRC stands for Class, Responsibilities, and Collaborators.
- Originally developed as a learning tool for making the transition to OO thinking.
- The goal was to provide the simplest possible conceptual introduction to OO modeling.
- A CRC card is a 3-x-5" or 4-x-6" lined index card.

class name	
subclasses:	
superclasses:	
Responsibilities	Collaborators

CRC – CLASS RESPONSIBILITY COLLABORATORS

- Requires people in two distinct roles: domain expert and object-oriented technology facilitator.
- Domain experts provide knowledge of the subject area
- OO facilitator coaches the team through the development of the cards and the eventual model.

CRC PROCESS

1. Before the Scenario Execution

   a. The Problem: Everyone agrees on the problem definition.

   b. Brainstorming for Classes: Based on the problem statement, the team identifies candidate classes using the vocabulary of the problem.

   c. Filtering Classes: The team works on definitions for each class, eliminating synonyms and conflicts.

   d. Assigning Cards: Each team member is assigned responsibility for one or more classes.

2. The Scenario Execution

   a. Each scenario expresses something that the system is supposed to do. The team walks through the scenario identifying the responsibilities of each class in the scenario.

   b. Each discovered responsibility is recorded on the card of the corresponding class.

CRC PROCESS

3. During the Scenario Execution

a. Grouping the Cards: The team identifies similar classes.

b. Scenario List: The team reviews the scenario coverage for completeness.

c. Collaboration Drawings: The cards are combined on a wall or white board to show how they cooperate in the execution of the scenarios.

4. After the Scenario Execution

a. The team reviews the resulting model and plans the implementation.

SAMPLE CRC CARD

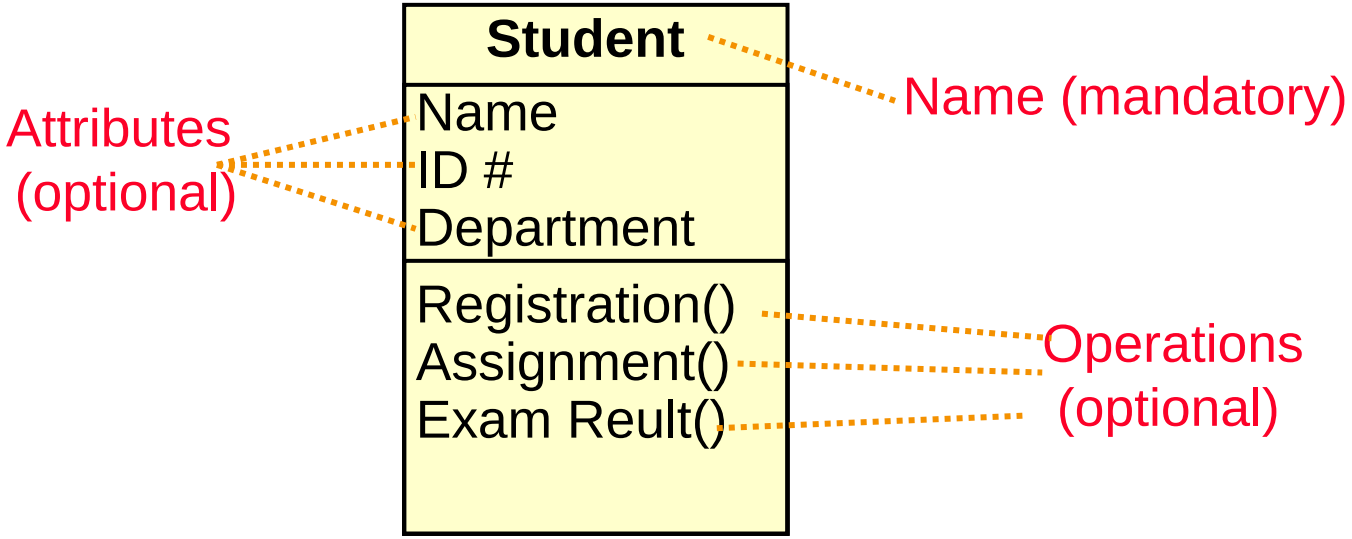
Class Name	**Account**
Description	Keeps track of owner, balance and transactions for each instance of customer's bank account
Superclass	
Subclass	Current Account, Saving Account
Responsibilities	**Collaborators**
Keep Track of Balance	Transaction
Update bank database	Database
Keep track of customer	Customer

CLASS DIAGRAM - INTRODUCTION

- A class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- These diagrams are the most common diagram found in modeling object-oriented systems.
- Class diagrams address the static design view of a system.
- Class diagrams that include active classes address the static process view of a system.
- Elements of a class diagram:
  - Class Notation
  - Relation Between Classes

CLASS

- A class is a definition for a *type of object.*
- A class is a template *of similar types of objects*.
- The class defines the *rules;* the objects express the *facts*.
- The class defines what *can be;* the object describes *what is*.
- *Classes can be abstract*



CLASS NOTATION

- The ***name compartment*** uniquely defines a class (a type of object) within a package. Consequently, classes may have the same name if they reside in different packages. This compartment may contain **Stereotypes** that helps to understand the type of class in the context of other classes with similar roles within the system.

- The ***attribute compartment*** contains all the data definitions.

- The ***operations compartment*** contains a definition for each behavior supported by this type of object.

ATTRIBUTE/OPERATION VISIBILITY IN CLASS NOTATION

- **Public (+)** visibility allows access to objects of all other classes.
- **Private (-)** visibility limits access to within the class itself. For example, only operations of the class have access to a private attribute.
- **Protected (#)** visibility allows access by subclasses. In the case of *generalizations* (inheritance), subclasses must have access to the attributes and operations of the superclass, or they cannot be inherited.
- **Package (~)** visibility allows access to other objects in the same package.

MODELING AN ATTRIBUTE

visibility / attribute name : data type = default value {constraints}

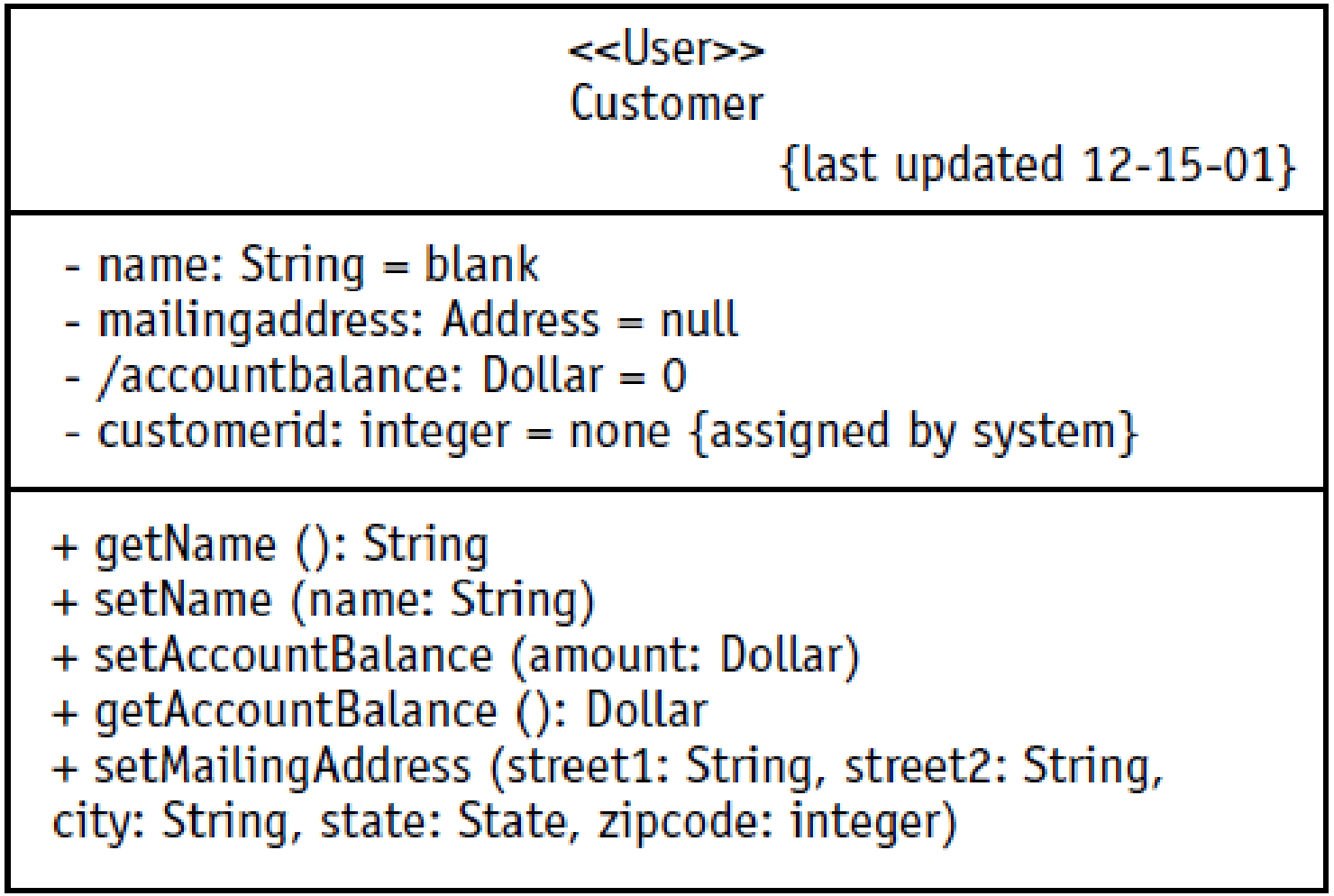
- **Visibility (+, -, #, ~):** Required before code generation
- **Slash (/):** The derived attribute indicator is optional. Derived values may be computed or figured out using other data and a set of rules or formulas.
- **Attribute name:** Required. Must be unique within the class.
- **Data type:** Required. This is a big subject. During analysis, the data type should reflect how the client sees the data. You could think of this as the external view.
- **Assignment operator and default value:** Optional. Default values serve two valuable purposes. First, default values can provide significant ease-of-use improvements for the client. Second and more importantly, they protect the integrity of the system from being corrupted by missing or invalid values.
- **Constraints:** Optional. Constraints express all the rules required to be fulfilled to access this attribute.
- **Class level attribute (underlined attribute declaration):** Optional. Denotes that all objects of the class share a single value for the attribute. (This is called a *static* value in Java.)

MODELING AN OPERATION

visibility operationName (argname : data type {constraints}, . . .) : return data type {constraints}

- **Visibility (+, -, #, ~):** Required before code generation.
- **Operation name:** Required. With ( ).
- **Parameter List/Arguments:** Any number of arguments is allowed in the parameter. Each argument requires an identifier and a data type. Constraints may be used to define the valid set of values that may be passed in the argument.
- **Return data type:** Required for a return value, but return values are optional.
- **Class level operation (underlined operation declaration):** Optional. Denoted as an operation accessible at the class level
- **Constraints:** Optional. In general, constraints express rules that must be enforced in the execution of the operation.

DRAWING CLASS NOTATION



Name Compartment

Attribute Compartment

visibility / attribute name : data type = default value {constraints}

Operation Compartment

visibility operationName (argname : data type {constraints}, ...) : return data type {constraints}

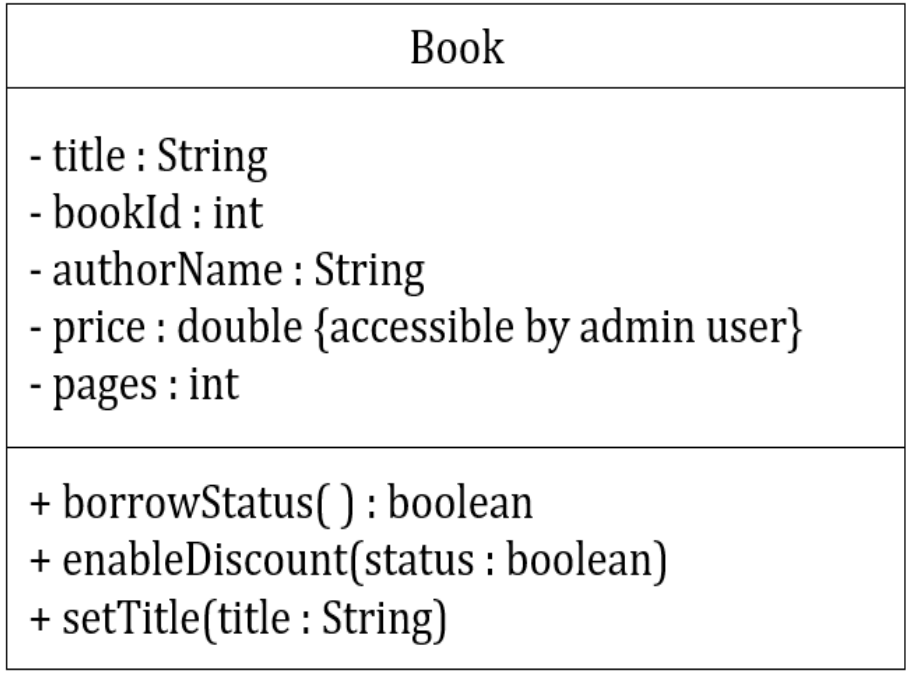
Figure 9-2 Complete class specification with all three compartments

PRACTICE CLASS NOTATION

In a system, there is a class named **Book**.

It has five attributes: **Title**, **BookId, AuthorName**, **Price,** and **Pages**. All the attributes are private and Price is only accessible by admin user. Title and AuthorName are String in type, BookId and Pages are in integer and Price is in double.

There are three methods in this class. They are: **borrowStatus**() which returns a boolean value, **EnableDiscount(boolean Status)** which returns nothing and **setTitle(String title)** which returns nothing. All the methods are public.



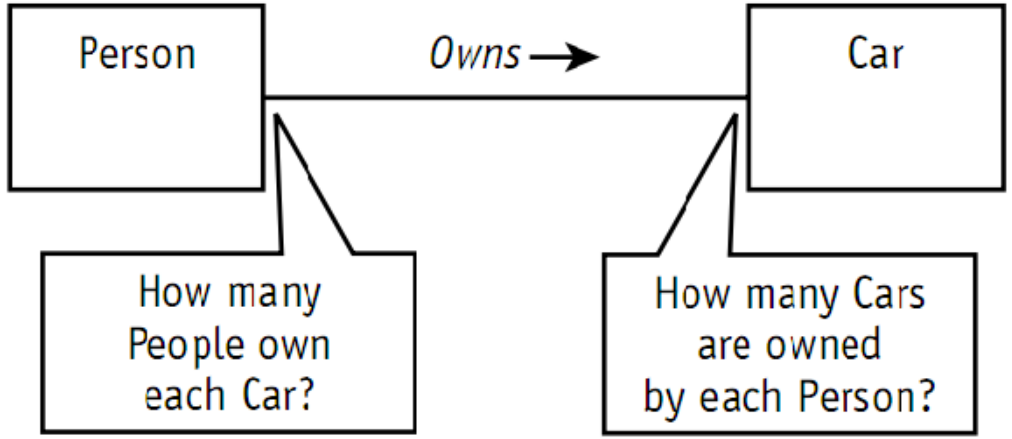
RELATIONSHIP IN CLASS DIAGRAM

- **Association -** a formal term for a type of relationship that this type of object may participate in. HAS A relationship.
  - Simple Association
  - Reflexive Association
  - Qualified Association
  - Association Class
  - Aggregate Association/ Aggregation
  - Composite Association/ Composition
- **Generalization –** When a class inherits another class. IS A relationship.
- **Realization –** When a class implements an interface.

ASSOCIATION

Associations may optionally have the following:

- **Association name**
  - Should be a verb or verb phrase;
- **Role names**
  - should be a noun or noun phrase describing the role;
- **Multiplicity**
  - The number of objects that can participate in a relation
- **Navigability**
  - The direction of Arrows

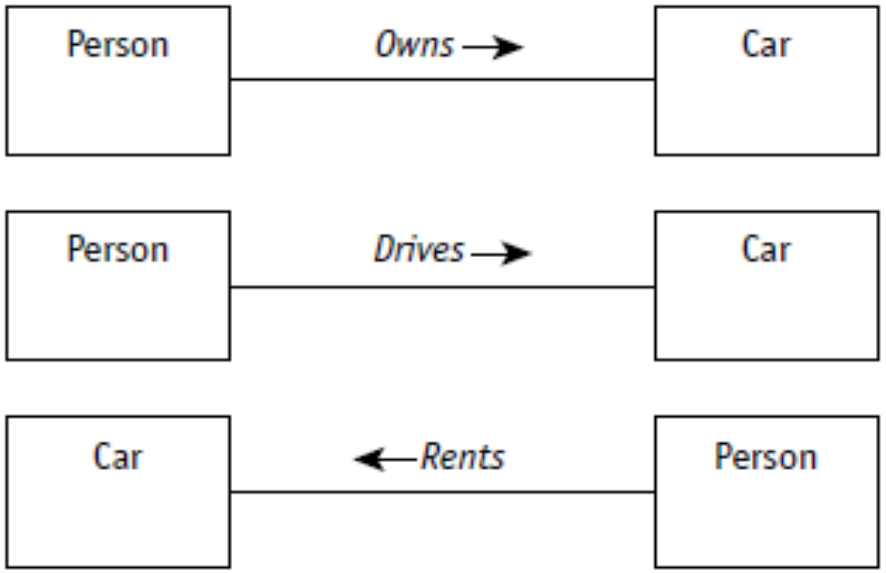


ASSOCIATION MULTIPLICITY

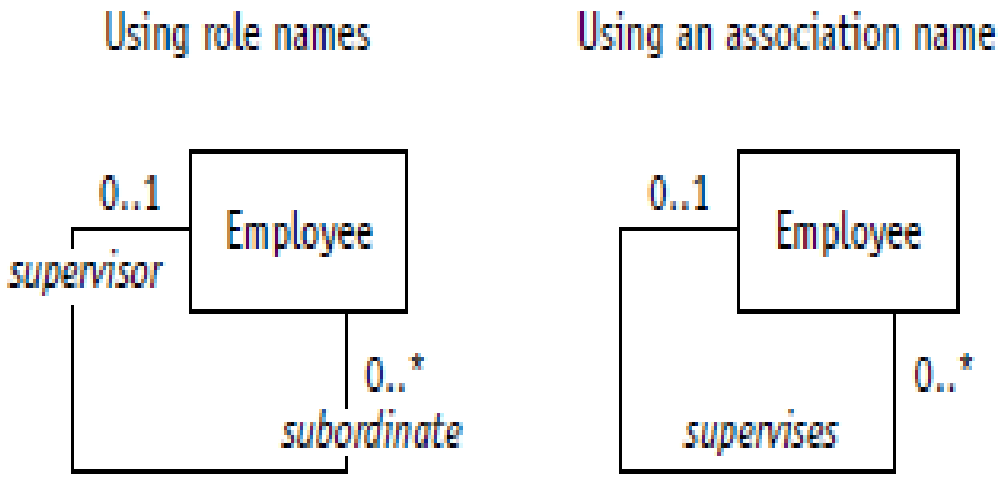
Exactly one	1
Many	*
Zero or more	0..*
One or more	1..*
Zero or one (optional association)	0..1
Specified range	2..4
Multiple, disjoint ranges	2, 4..6, 8

ASSOCIATION

Simple Association

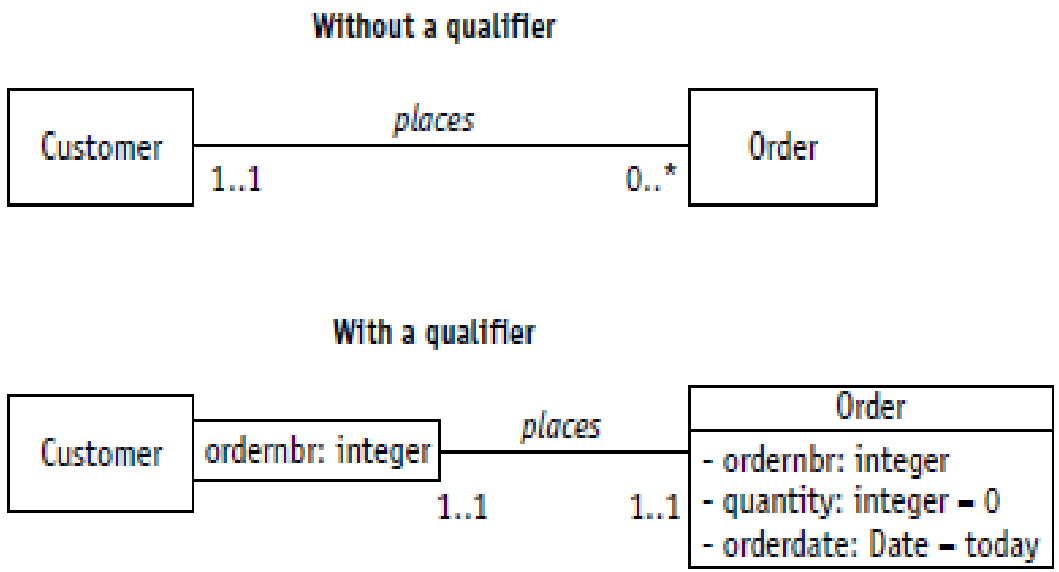


Reflexive Association

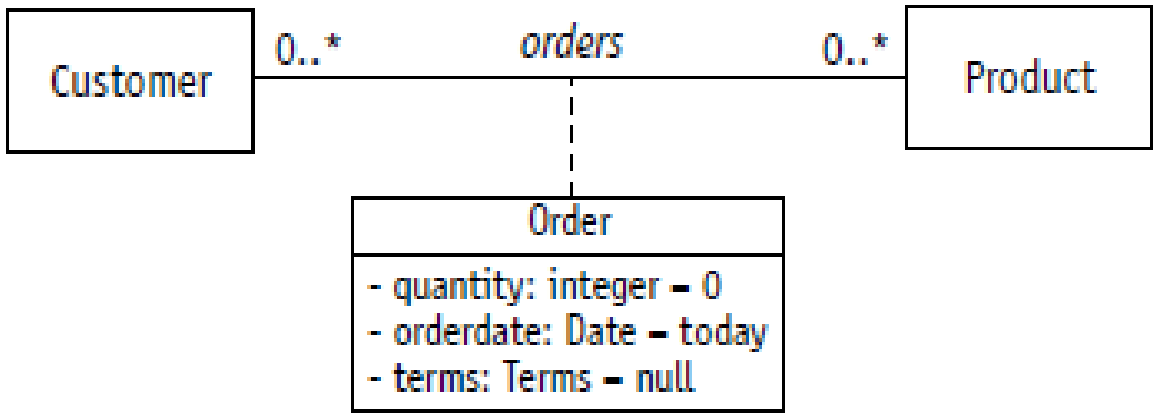


ASSOCIATION

Qualified Association

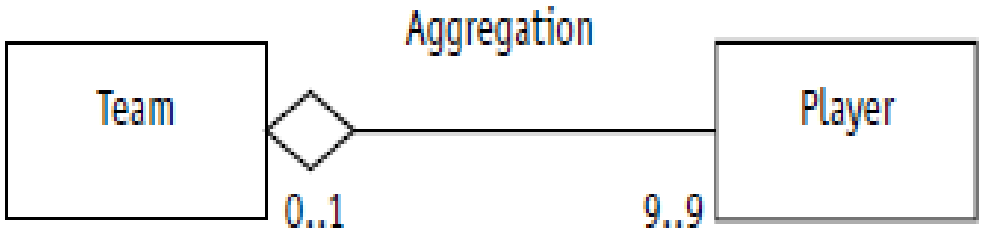


Association Class

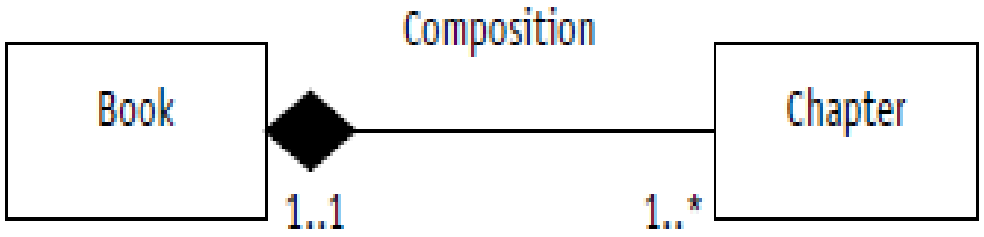


ASSOCIATION

- **Aggregate Association** – Whole-Part Relationship



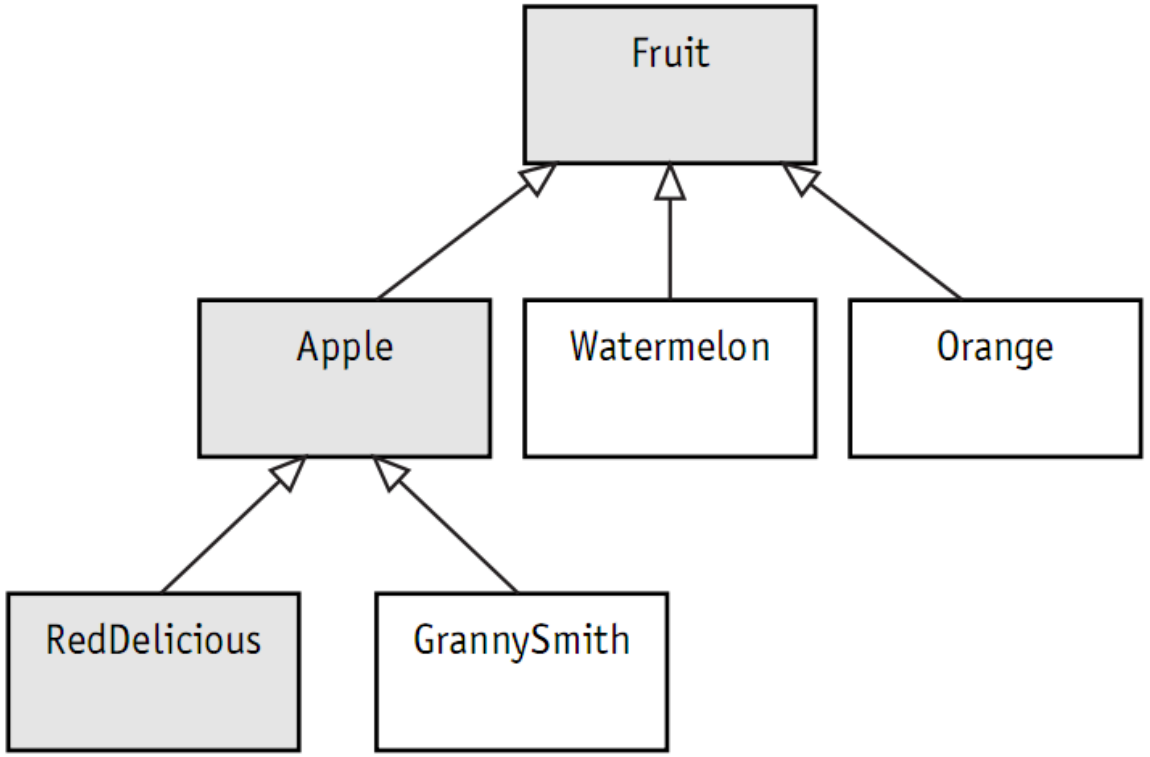
- **Composite Association** – Special Type of Aggregation where the life span of the part depends on the life span of the aggregate



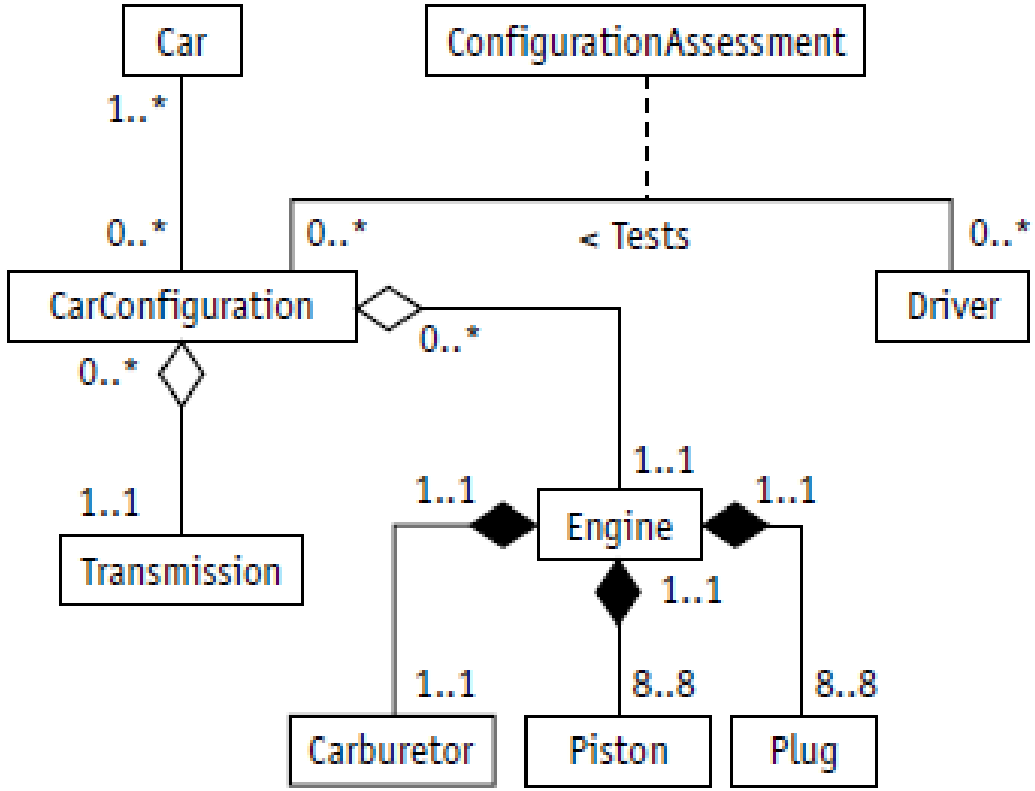
GENERALIZATION

- Generalization is the process of organizing the properties of a set of objects that share the same purpose.

- A generalization is not an association.



EXAMPLE

Problem statement: "Our Company maintains a group of race cars. Our cars use some of our new 8-cylinder engines and new transmissions. Once the engines are assembled, the pistons, carburetor, and plugs cannot be swapped between engines due to changes caused by the high temperatures.

"We want to keep records of the performance achieved by each engine in each car and each transmission in combination with each engine. Our drivers evaluate each car to give us their assessment of the handling. We need a system to track the configurations and the drivers' assessments."